

FRAUD360

Developer Environment Setup

Running, testing and extending Fraud360 on a developer machine

Version 1.1 · 2 October 2026

Finverge · Confidential — for the engineering team

Document control

Item	Detail
Document	Fraud360 — Developer Environment Setup Guide
Version	1.1 — 2 October 2026 (supersedes 1.0)
Companions	Infrastructure Requirements v1.1 and Deployment Instructions v1.1 (servers); the repository README.md (API tables, feature notes)
Changes in 1.1	Roles and permissions are data (new §5.5 and §8.5; §8.4 rewritten); test setup changes (§7.1, §7.2); new troubleshooting rows (§9); the React console proxy note updated.
Audience	Engineers joining the Fraud360 code base; QA engineers who need a local stack
Basis	Commands were checked against the repository on 2 October 2026, including full test-suite runs under the project virtual environment (about 1,520 tests; the last full run had one finding, in the old console’s static check, since fixed) and a start-up of the stack against the local database to sign in as each role; §9 lists the traps found while preparing it

Contents

1. What you will have at the end.....	4
2. Prerequisites.....	5
3. Get the code and find your way around.....	6
4. Backend setup	7
4.1 Python environment.....	7
4.2 PostgreSQL role and database.....	7
4.3 Local configuration (`.env`).....	7
4.4 Create the schema and the service roles.....	8
5. Run the stack	9
5.1 Start everything	9
5.2 First sign-in.....	9
5.3 Demo data	9
5.4 Workers	10
5.5 Roles and permissions in development.....	10
6. Front-end work and other platforms.....	11
6.1 The shipped console	11
6.2 The React console.....	11
6.3 Linux and macOS.....	11
7. Tests and quality checks	12
7.1 Running the suite.....	12
7.2 Writing tests that survive	12
8. Everyday workflows.....	13
8.1 Changing a model	13
8.2 Adding a Lane B (near-real-time) rule	13

8.3 Adding a Lane C signal 13

8.4 Adding an API endpoint or service 13

8.5 Gating an action by role..... 13

8.6 Rebuilding the design documents 14

8.7 Resetting your environment..... 14

9. Troubleshooting and known traps..... 15

Appendix A — Cheat sheet 16

1. What you will have at the end

A working local copy of the platform: PostgreSQL with the schema applied, all application services running with auto-reload, the console open in a browser with demo tenants and synthetic data, the detection and Lane C workers running when you need them, and the test suite green on your machine. Allow about an hour the first time, most of it downloads (PyTorch's CPU wheel is a few hundred MB).

Part	What it is	Needed for
Backend	Nine FastAPI services on ports 8080–8088, one Python code base	Everything
PostgreSQL	One database, ten schemas, one login role per service	Everything
Shipped console	Static single-page app served by the gateway on :8080 (no build step)	The console most users see today
React console	<code>frontend/</code> — Vite + React + Tailwind, dev server on :5180 (migration in progress)	Only if you work on the new console
Workers	Detection, Lane C, notifications, file intake: scripts under <code>scripts/</code>	Anything involving scoring, statements or alerts
Tests	pytest suite against a throwaway <code>cp_test</code> database	Every change

2. Prerequisites

Tool	Version	Why	Install (Windows, winget)
Windows 10/11	—	The helper scripts (<code>run_local.ps1</code> , <code>migrate.ps1</code> , <code>run.bat</code>) are PowerShell. Linux/macOS work: \$6.3 gives the equivalent commands	—
Python	3.12	Matches the production image (<code>python:3.12-slim</code>); other versions are untested	<code>winget install Python.Python.3.12</code>
PostgreSQL	16 (14+ works)	The only data store. Install it natively; Docker is not needed	<code>winget install PostgreSQL.PostgreSQL.16</code>
Git	current	Source control	<code>winget install Git.Git</code>
Node.js	20 LTS or later (24 is in use)	React console only	<code>winget install OpenJS.NodeJS.LTS</code>
Tesseract OCR	5.x	Lane C scanned-statement OCR path. Put <code>tesseract.exe</code> on <code>PATH</code>	<code>winget install UB-Mannheim.TesseractOCR</code>
Ollama	optional	Only for the opt-in LLM features (narrative polish, AI Insights, Lane C qualitative read). All are off by default	<code>ollama.com</code>

Machine: 16 GB RAM is comfortable. Nine Python processes, PostgreSQL and the browser are fine on 8 GB, but the analytics service loads NumPy and scikit-learn, and Lane C or ring-scoring work loads PyTorch.

About Docker

`docker compose up --build` is described in the README, but `docker-compose.yml` lags the current service set (it has five services; the ingestion, decision, notification, Lane C and ISO 8583 services are missing). Use the native route in this guide. Refreshing the container assets is a known open item.

3. Get the code and find your way around

```
git clone <repository-url> Fraud360
cd Fraud360
```

The repository is plain git on **main**. Everything below is run from the repository root. Python commands use the project virtual environment, never the system Python (see §9, trap 1).

Path	What lives there
<code>services/gateway/</code>	Edge: JWT check, rate limit, <code>/api/*</code> proxy, serves the shipped console from <code>app/static/</code>
<code>services/tenant_service/</code>	Tenants, users, MFA, SSO (OIDC/SAML), sessions, machine credentials; seeds the first admin at startup
<code>services/branding_service/</code> , <code>config_service/</code>	White-label themes; versioned per-tenant rules; the rule catalogue (<code>ews_catalogue.py</code>)
<code>services/ingestion_service/</code>	Rail adapters, file/ISO 20022 intake, CBS events, the queue detection consumes
<code>services/analytics_service/</code>	Dashboards, detection engine (<code>app/detection/</code>), cases, filings, board packs, reference feeds, metering
<code>services/decision_service/</code>	Lane A inline <code>POST /decide</code>
<code>services/lane_c_service/</code>	Borrower statement review: PDF extraction, signals (<code>app/signals/</code>), scoring
<code>services/notification_service/</code> , <code>iso8583_gateway/</code> , <code>demo_idp/</code>	Delivery channels; ISO 8583 TCP front door (not started by default); a demo identity provider for SSO testing
<code>packages/cp_common/</code>	Shared library: settings, DB, auth, tenancy, MFA, crypto, observability, <code>observations.py</code> (the signal feature hook), <code>permissions.py</code> (the catalogue of gated actions), <code>rbac.py</code> (starter role templates only) and <code>dynamic_roles.py</code> (how every service reads a tenant's roles)
<code>alembic/</code> , <code>alembic.ini</code> , <code>migrate.ps1</code>	Schema migrations — the only thing that creates tables
<code>scripts/</code>	Workers (<code>run_*.py</code>), seeders, load test, model training, <code>provision_db_roles.py</code>
<code>frontend/</code>	React console (separate toolchain)
<code>tests/</code>	pytest suite (about 100 files)
<code>docs/</code>	BRD/FSD/HLD sources, <code>rbi_ews_mapping.py</code> (RBI-42 coverage), <code>make_pdf.py</code> , design notes
<code>tools/</code>	<code>jscheck.py</code> , <code>htmlcheck.py</code> — static checks for the shipped console

Architectural rules to know on day one. Each service is layered (routes → services → repositories → models). A service reaches only its own schema (plus **platform**); there are no cross-service foreign keys and no cross-service SQL, enforced by the database roles, not by convention. Analytics reading the ingestion queue is the one deliberate exception. Cross-service needs go over HTTP.

4. Backend setup

4.1 Python environment

```
python -m venv .venv
.\.venv\Scripts\python -m pip install --upgrade pip
.\.venv\Scripts\python -m pip install -r requirements.txt
.\.venv\Scripts\python -m pip install -e packages/cp_common
```

`requirements.txt` carries its own extra index line for the **CPU-only** PyTorch wheel, so a plain install is correct. Verify, because a wrong resolve is expensive:

```
.\.venv\Scripts\python -c "import cp_common, torch; print(cp_common.__file__); print(torch.__version__, torch.cuda.is_available())"
# expect: a path inside THIS checkout, then something like 2.14.0+cpu False
```

In VS Code or PyCharm select `.venv\Scripts\python.exe` as the interpreter so test discovery and debugging use it.

4.2 PostgreSQL role and database

As the `postgres` superuser. The examples use `postgres/postgres` and `cp/cp_password`, which are the defaults the test fixtures and scripts assume; change them if your local server differs, and override with the environment variables named in §7.

```
psql -U postgres -c "CREATE ROLE cp LOGIN PASSWORD 'cp_password';"
psql -U postgres -c "CREATE DATABASE controlplane OWNER cp;"
```

`cp` is the **owner** role: it owns the schemas and runs migrations. The services will connect as their own, narrower roles, created in §4.4.

4.3 Local configuration (`.env`)

Settings are read from the environment, then from a `.env` file in the working directory. Copy the template and fix three things:

```
copy .env.example .env
```

Setting	Do this	Why
<code>DATABASE_URL</code>	Change host <code>postgres</code> to <code>localhost</code>	The template targets the Docker network name. Scripts run outside <code>run_local.ps1</code> read this file directly
<code>CP_ENCRYPTION_KEYS</code>	Add a generated key (below)	Secrets (MFA seeds, IdP client secrets, feed credentials) are sealed; with no key, saving one raises an error
<code>SEED_DEMO_TENANT</code>	Leave <code>true</code> for a first look, or <code>false</code> and use the seeders	Controls the <code>demo-bank</code> tenant created on first boot

```
.\.venv\Scripts\python -c "import os,base64;
print('CP_ENCRYPTION_KEYS=1:' +base64.b64encode(os.urandom(32)).decode())"
# paste the printed line into .env
```

Do not use `CP_ALLOW_PLAINTEXT_SECRETS` to avoid generating a key; it exists for throwaway tests only. `.env` is for your machine and must not be committed. `run_local.ps1` sets its own database URLs and service URLs as process environment variables, which take precedence over `.env`.

4.4 Create the schema and the service roles

Order matters: migrations create the schemas, and the role script grants on them.

```
.\migrate.ps1                # alembic upgrade head, as the owner role
.\migrate.ps1 current        # confirm it prints the head revision

.\.venv\Scripts\python scripts\provision_db_roles.py `
  --superuser-url postgresql+psycopg2://postgres:postgres@localhost:5432/controlplane
```

This creates `svc_tenant`, `svc_branding`, `svc_config`, `svc_analytics`, `svc_ingestion`, `svc_notify`, `svc_decision` and `svc_lane`, each limited to its own schema plus `platform`, all with the development password `cp_password` that `run_local.ps1` expects. Re-run it whenever a migration adds a **schema** (new tables in an existing schema are covered automatically).

5. Run the stack

5.1 Start everything

```
.\run_local.ps1          # or double-click / run: run.bat
```

The script first frees ports 8080–8088 (matched by port, so it never kills an unrelated `python.exe`), then starts each service with `uvicorn --reload` under its own database role, and stops them all on Ctrl+C. All service logs interleave in that one window.

Service	Port	Swagger	DB role
gateway + shipped console	8080	—	none
tenant-service	8081	:8081/docs	svc_tenant
branding-service	8082	:8082/docs	svc_branding
config-service	8083	:8083/docs	svc_config
analytics-service	8084	:8084/docs	svc_analytics
ingestion-service	8085	:8085/docs	svc_ingestion
decision-service (Lane A)	8086	:8086/docs	svc_decision
notification-service	8087	:8087/docs	svc_notify
lane-c-service	8088	:8088/docs	svc_lanec

The ISO 8583 gateway is **not** started: it needs a machine credential and an explicit fail-open/closed decision. Start it by hand only when working on it.

5.2 First sign-in

Open <http://localhost:8080>. The first platform administrator is seeded by tenant-service at startup from `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` (the documented development defaults are `admin@finverge.local` / `ChangeMe123!`). Unless you set `SEED_ADMIN_MUST_CHANGE_PASSWORD=false`, the first sign-in forces a password change; the password policy requires 12+ characters with upper, lower, digit and symbol and rejects the email local part.

5.3 Demo data

```
# tenants, branding, default rules (through the real onboarding flow; idempotent)
.\.venv\Scripts\python scripts\api_seed.py

# one user per role on a tenant, via the real invite + forced-reset flow
.\.venv\Scripts\python scripts\seed_demo_users.py --tenant hdfc-demo

# realistic, internally consistent transactions, alerts and cases with injected rings
.\.venv\Scripts\python scripts\generate_synthetic_data.py --reset --days 60 --txns 9000
.\.venv\Scripts\python scripts\generate_synthetic_data.py --append --minutes 30      # live-feeling top-up
```

`api_seed.py` and `seed_demo_users.py` go through the gateway, so the stack must be running; they read `GATEWAY_URL`, `SEED_ADMIN_EMAIL` and `SEED_ADMIN_PASSWORD` if you changed the defaults. Demo credentials are defined in the seed scripts themselves, not repeated here. The gateway rate-limits each user to 20 requests/s (burst 40), so a seeder or a tight front-end loop can trip a 429; that is the limiter working, not a bug.

5.4 Workers

The services alone accept data but do not score it. Run the workers for the feature you are touching, each in its own terminal, with the owner connection (simplest in development; the per-service roles matter in staging and production):

```
$env:PYTHONPATH = (Get-Location).Path
$env:DATABASE_URL = "postgresql+psycopg2://cp:cp_password@localhost:5432/controlplane"
.\.venv\Scripts\python scripts\run_detection.py --loop 10           # score queued transactions
.\.venv\Scripts\python scripts\run_lane_c.py --loop 30            # parse + score uploaded statements
.\.venv\Scripts\python scripts\run_notifications.py --loop 60     # case clocks, returns, alerts
.\.venv\Scripts\python scripts\run_file_intake.py --once          # SFTP-style drops (needs
INGEST_DROP_ROOT)
```

If you are working on...	Run
Detection rules, alerts, cases, dashboards from live ingestion	ingestion API + <code>run_detection.py</code> (and <code>live_feed.py</code> or <code>loadtest.py</code> to generate traffic)
Lane C (statement upload, signals, scoring)	lane-c service + <code>run_lane_c.py</code> ; Tesseract on <code>PATH</code> for scanned PDFs
Reference lists, board packs, retention, usage roll-up, ring scores	the matching <code>run_*.py</code> / <code>refresh_ring_scores.py</code> once, on demand (dry-run flags are the default for retention)
Lane A	decision-service only; get a machine token from <code>/machine/token</code> and call <code>/decide</code> (see the Integration Handshaking Specification)
SSO	<code>demo_idp</code> (set <code>DEMO_IDP_ENABLED=true</code> ; it is deliberately inert otherwise) plus <code>scripts/seed_demo_idp.py</code>

5.5 Roles and permissions in development

Who may do what is stored in the database, per tenant, and edited in the console; no service decides by role name. Onboarding (and the migrations, for tenants that already exist) copy ten starter roles into each tenant as ordinary rows. From then on you can add a role, change what any role may reach or do, or delete one, from the **Roles** page as a tenant administrator, or as the platform administrator for any tenant. An edit that takes something away applies on that person's next request; giving a role more is staged until a **different** administrator confirms it, so a local tenant needs two administrators to try that path.

Three practical consequences while developing: a tenant with no role rows can do nothing (re-run `.\migrate.ps1` after a database reset); a role you change shows in `GET /auth/me`, which is what the console builds its menu and Home page from; and the platform operator (`platform_admin`) is the one role that is not a row. The permission catalogue and the starter grants are in `packages/cp_common/cp_common/permissions.py` and `rbac.py`.

6. Front-end work and other platforms

6.1 The shipped console

Plain HTML/JS/CSS under `services/gateway/app/static/`. No build step: edit, refresh. Assets are served `no-cache`, so a hard refresh is not needed. Run the static checks before committing (a broken string literal once blanked the entire console):

```
.\.venv\Scripts\python tools\jscheck.py
```

6.2 The React console

```
cd frontend
npm install          # or npm ci with the lockfile
npm run dev          # Vite on http://localhost:5180
npm run lint         # oxlint
npm run build        # tsc -b && vite build (type-checks the whole app)
```

API proxy

The dev server proxies `/api` to the gateway at `http://127.0.0.1:8080`, which is where `run_local.ps1` starts it, so start the backend first. To use a gateway elsewhere, set `FRAUD360_GATEWAY_URL` before `npm run dev` (for example `$env:FRAUD360_GATEWAY_URL='http://127.0.0.1:8090'`). Sign in against the gateway you proxy to; the console is a client of the same `/api/*` surface as the shipped one.

When type-checking by hand use the project config (`tsc -p tsconfig.app.json --noEmit`): a bare `tsc --noEmit` with the solution-style config checks zero files and reports success.

6.3 Linux and macOS

Nothing in the application is Windows-specific; only the helper scripts are. Equivalent steps:

```
python3.12 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt && pip install -e packages/cp_common
export PYTHONPATH=$PWD DATABASE_URL=postgresql+psycopg2://cp:cp_password@localhost:5432/controlplane
python -m alembic upgrade head
python scripts/provision_db_roles.py --superuser-url
postgresql+psycopg2://postgres:postgres@localhost:5432/controlplane

# start one service (repeat per row of the table in 5.1, each with its own role and search path):
DATABASE_URL=postgresql+psycopg2://svc_tenant:cp_password@localhost:5432/controlplane
DB_SEARCH_PATH=tenant,platform \
  uvicorn services.tenant_service.app.main:app --port 8081 --reload
# service URL variables (TENANT_SERVICE_URL ... LANE_C_SERVICE_URL) must point at 127.0.0.1:<port>; see
run_local.ps1 for the full list
```

7. Tests and quality checks

7.1 Running the suite

```
.\.venv\Scripts\python -m pytest -q          # everything
.\.venv\Scripts\python -m pytest tests\test_cbs_events.py -q  # one file
.\.venv\Scripts\python -m pytest -k lane_c -q  # by name
.\.venv\Scripts\python -m pytest --collect-only -q -p no:warnings | more  # see what exists
.\migrate.ps1 check          # models vs migrations: expect no diff
.\.venv\Scripts\python tools\jscheck.py      # shipped-console static check
```

About 1,520 tests are collected today, and a full run takes roughly half an hour. (The README’s “646 tests” is out of date.) The fixtures in `tests/conftest.py` connect as the `postgres` superuser, **drop and recreate a database named `cp_test`**, run every Alembic migration into it, and then run the tests, so the migrations are tested as well as the models and your development data is never touched. Your development stack can stay running.

Variable	Default	Use
TEST_ADMIN_URL	postgresql+psycopg2://postgres:postgres@localhost:5432/postgres	Superuser connection used to create and drop the test database
TEST_DB_NAME	cp_test	Set this to a name of your own whenever anyone else may be running tests on the same server (another terminal, another session, CI): every session drops and recreates its database, and a shared name makes runs fail with “database does not exist”
TEST_DATABASE_URL	...cp:cp_password@localhost:5432/cp_test	Owner connection to the test database

7.2 Writing tests that survive

- **Prove a check can fail.** The reconciliation suite deliberately corrupts rows and asserts the right invariants trip; a suite that only ever passes proves nothing. Do the same for new signals: test the clean case, the breach, and the “unmeasurable” case (a signal with no data must say so, never report clean).
- **Do not hard-code dates.** Use dates relative to now; tests with fixed calendar dates have gone stale on their own.
- **Tests that pin a catalogue break when the catalogue grows.** `test_reference_data.py` asserts which rules need which reference feed, and the declared-signal sets in `docs/rbi_ews_mapping.py`. When you add a rule or signal, expect to update them, and that is the point.
- **Use the existing fixtures** (`seeded`, the per-service client fixtures, `tid`) rather than seeding by hand; they set up tenants, roles and data consistently.
- **A tenant needs role rows.** Roles are the only authority, so a test that inserts a tenant directly must call `seed_default_roles(db, tenant.id)` (the shared `seeded` fixture does), or nobody in it can do anything. An autouse fixture routes every service’s role lookup to the in-process tenant-service; tests that change a role should assert the very next request obeys it.
- **Gate the action, then test both ways.** For a new gated action, test a role that holds it, one that does not, and that an administrator granting or removing it changes the outcome. `tests/test_screening_permissions.py` and `tests/test_roles_are_data.py` are the patterns.
- There is no CI workflow in the repository today (the README mentions one). Run the suite and `migrate.ps1 check` yourself before every commit.

8. Everyday workflows

8.1 Changing a model

```
# edit the model, then:
.\migrate.ps1 revision --autogenerate -m "what changed"
# READ the generated file in alembic/versions/ - autogenerate misses renames and server defaults
.\migrate.ps1                                # apply
.\migrate.ps1 check                          # must report no differences
# a NEW SCHEMA (not just a new table): re-run provision_db_roles.py
```

Commit the model change and its migration together. `AUTO_CREATE_TABLES` must stay `false` so models and migrations cannot silently diverge.

8.2 Adding a Lane B (near-real-time) rule

- Declare it in `services/config_service/app/ews_catalogue.py` (`_RULES`; add to `CREDIT_LINKED_RULES` if it is a credit signal).
- Compute its feature in the detection path: `packages/cp_common/cp_common/observations.py` for transaction-level features, or `services/analytics_service/app/detection/cbs_features.py` for CBS loan-conduct signals (add the event kind to `FEEDS`).
- If it depends on an external list, register the feed in `reference_model.py` (`LIST_KINDS`) and `features.py` (`NEEDS_REFERENCE_DATA`); the rule then reports “unmeasurable” until a tenant loads the list.
- Add tests (breach, no-breach, no-data), update the catalogue-pinning assertions, and, if it closes an RBI signal, update `docs/rbi_ews_mapping.py` and run `python docs/rbi_ews_mapping.py` to confirm the coverage line.

8.3 Adding a Lane C signal

- Write a pure function in `services/lane_c_service/app/signals/` returning a severity and evidence; keep I/O out of it.
- Wire it into `signal_engine.compute_all(...)` and the I/O helpers in `runner.py`; list it in `signal_catalogue.py`.
- Severity bands are fixed (`band()`: 50+ critical, 30+ high, 15+ medium, 5+ low). Never fabricate a clean result when the input is missing: use the `_unmeasurable()` pattern.
- Test the function in isolation and one API round trip.

8.4 Adding an API endpoint or service

Follow the existing layering. Put routes in `routes/`, logic in `services.py`, SQL in `repositories.py`. Scope every endpoint with `resolve_tenant_scope` and audit state changes with `record_audit`. Decide who may call it with a `permission`, never a role name (see §8.5): `has_permission(tenant_id, principal.role, "your.permission")` from `cp_common.dynamic_roles`. A new `service` also needs: an entry in `schemas_db.py` (schema, owner, grants), a role in `provision_db_roles.py`, an entry in `run_local.ps1`, a gateway route if the console calls it, `TENANT_SERVICE_URL` and `INTERNAL_API_KEY` in its environment (it reads roles from tenant-service), and `install_health` / `install_observability` like its neighbours.

8.5 Gating an action by role

There is deliberately nowhere to write “these roles may do this” in code. To gate an action:

- Add it to `PERMISSIONS` in `packages/cp_common/cp_common/permissions.py` with a group and a label; the console’s role editor offers it automatically.

- Guard the endpoint with `has_permission(...)` (or, for tenant-service, `roles.capability(...)`), returning 403 `role_not_permitted`. Reading and changing are usually separate permissions.
- Decide who holds it by default: add it to the starter template in `rbac.py` (`_STARTER`) and write a **new** Alembic data migration that grants it to the existing tenants' starter-role rows (see `f3c6d9e2a5b8_screening_and_credit_health_permissions.py`). Never edit an applied migration, and never overwrite a permission a tenant has already edited.
- Case-workflow actions follow a pattern: one permission per transition, named `case.act.<action>`; a test keeps the workflow and the catalogue identical.
- If a page depends on it, hide the page from `me.permissions` in `frontend/src/lib/access.ts`; that is only the menu, the server check is the control.

What stays in code

The catalogue of actions that exist (it has to name them to guard them), the starter templates a new tenant is seeded from, and the platform operator (Finverge staff). None of these is consulted to allow a request for a tenant's own roles.

8.6 Rebuilding the design documents

BRD, FSD and HLD are HTML under `docs/` (`brd.html`, `fsd.html`, `hld.html`) rendered by `docs/make_pdf.py` using headless Edge. Run it from **PowerShell**, not Git Bash (it fails silently there). Version numbers live in two places, the HTML and the `jobs` list in `make_pdf.py`; change both. Docs and code stay in step: a feature that changes a signal count or a contract updates the documents in the same piece of work.

8.7 Resetting your environment

```
# stop run_local.ps1 first (Ctrl+C), then:
psql -U postgres -c "DROP DATABASE controlplane;"
psql -U postgres -c "CREATE DATABASE controlplane OWNER cp;"
.\migrate.ps1
.\.venv\Scripts\python scripts\provision_db_roles.py --superuser-url
postgresql+psycopg2://postgres:postgres@localhost:5432/controlplane
.\run_local.ps1      # then re-seed (5.3)
```

9. Troubleshooting and known traps

Trap 1 — the wrong Python imports a stale copy of the shared library

On a machine that has ever had an older checkout installed, the system `python` may import `cp_common` from a different folder, producing errors such as `cannot import name 'LANE_C' from cp_common.schemas_db` or `'Settings' object has no attribute ....` This was reproduced while preparing this guide: collection under the system Python failed with 9 errors, and under `.venv` succeeded with 1,486 tests. Always run `.\venv\Scripts\python`, and check `python -c "import cp_common; print(cp_common.__file__)"` points into your checkout.

Symptom	Cause	Fix
<code>ModuleNotFoundError: joblib</code> or <code>torch</code> in tests	Not using the project venv, or a partial install	Use <code>.venv</code> ; re-run §4.1
<code>relation "..."</code> does not exist from a service	Migrations not applied, or the service's <code>DB_SEARCH_PATH</code> / role wrong	<code>.\migrate.ps1 current</code> ; start via <code>run_local.ps1</code>
<code>permission denied for schema ...</code>	A new schema, or roles never provisioned	Re-run <code>provision_db_roles.py</code>
Scripts fail to resolve host <code>postgres</code>	<code>.env</code> still has the Docker host name	Change it to <code>localhost</code> (§4.3)
Port already in use	A previous run still holds it	<code>run_local.ps1</code> frees 8080–8088 itself; for others <code>Get-NetTCPConnection -LocalPort <p></code>
Intermittent ~2 s delays between services	<code>localhost</code> resolves to <code>::1</code> first on Windows	Use <code>127.0.0.1</code> in service URLs (what <code>run_local.ps1</code> does)
HTTP 429 from the gateway	Per-user rate limit (20/s, burst 40) hit by a loop or seeder	Throttle the caller; memoise front-end filter objects
Saving an MFA seed or IdP secret raises a key error	No <code>CP_ENCRYPTION_KEYS</code>	§4.3
Forced password change blocks scripts	<code>SEED_ADMIN_MUST_CHANGE_PASSWORD=true</code>	Set it <code>false</code> in your local <code>.env</code> for scripted runs only
Statement upload never turns into signals	Lane C worker not running, or Tesseract missing for a scanned PDF	Run <code>run_lane_c.py --loop</code> ; <code>tesseract --version</code>
Alerts never appear for ingested transactions	No detection worker	Run <code>run_detection.py --loop 10</code>
Location panel empty	Geolocation uses the public <code>ip-api.com</code> by default and may be blocked or rate-limited	Expected offline; set <code>GEOLOCATE_URL</code> to something internal if you need it
React console shows 502/503 on <code>/api</code>	Backend not running, or <code>FRAUD360_GATEWAY_URL</code> points elsewhere	Start <code>run_local.ps1</code> ; see §6.2
<code>make_pdf.py</code> exits silently	Run from Git Bash	Use PowerShell
Everyone gets 403 <code>role_not_permitted</code> after a database reset	No role rows for the tenant (or the service cannot reach tenant-service for role lookups)	<code>.\migrate.ps1</code> (it seeds the roles); check <code>TENANT_SERVICE_URL</code> for the failing service
Tests fail with “database <code>cp_test</code> does not exist”	Another run (a second terminal or session) dropped the shared test database	Set <code>TEST_DB_NAME</code> to your own name for your runs
A role edit seems ignored	It raised a privilege and is staged until a different administrator confirms	Look for “pending confirm” on the Roles page; confirm as another administrator
Test run slow to start	It rebuilds <code>cp_test</code> through all migrations every session	Expected; keep your stack running separately

Appendix A — Cheat sheet

Task	Command
Start the stack	<code>.\run_local.ps1</code> (or <code>run.bat</code>)
Apply migrations	<code>.\migrate.ps1</code>
New migration	<code>.\migrate.ps1 revision --autogenerate -m "..."</code>
Model/migration drift	<code>.\migrate.ps1 check</code>
All tests	<code>.\.venv\Scripts\python -m pytest -q</code> (set <code>TEST_DB_NAME</code> first if anyone else runs tests)
Seed tenants	<code>python scripts\api_seed.py</code>
Seed users	<code>python scripts\seed_demo_users.py --tenant hdfc-demo</code>
Synthetic data	<code>python scripts\generate_synthetic_data.py --reset --days 60 --txns 9000</code>
Detection worker	<code>python scripts\run_detection.py --loop 10</code>
Lane C worker	<code>python scripts\run_lane_c.py --loop 30</code>
Load test	<code>python scripts\loadtest.py --tenant <slug> --rate 200 --duration 60</code>
RBI-42 coverage	<code>python docs\rbi_ews_mapping.py</code>
Service health	<code>curl http://127.0.0.1:8084/health/ready</code>
Metrics	<code>curl -H "X-Internal-Key: <key>" http://127.0.0.1:8084/metrics</code>
React dev server	<code>cd frontend; npm run dev</code> (<code>http://localhost:5180</code>)

(Prefix Python commands with `.\.venv\Scripts\` on Windows. Scripts that talk to the database directly need `PYTHONPATH` set to the repository root and a `DATABASE_URL` that points at `localhost`.)